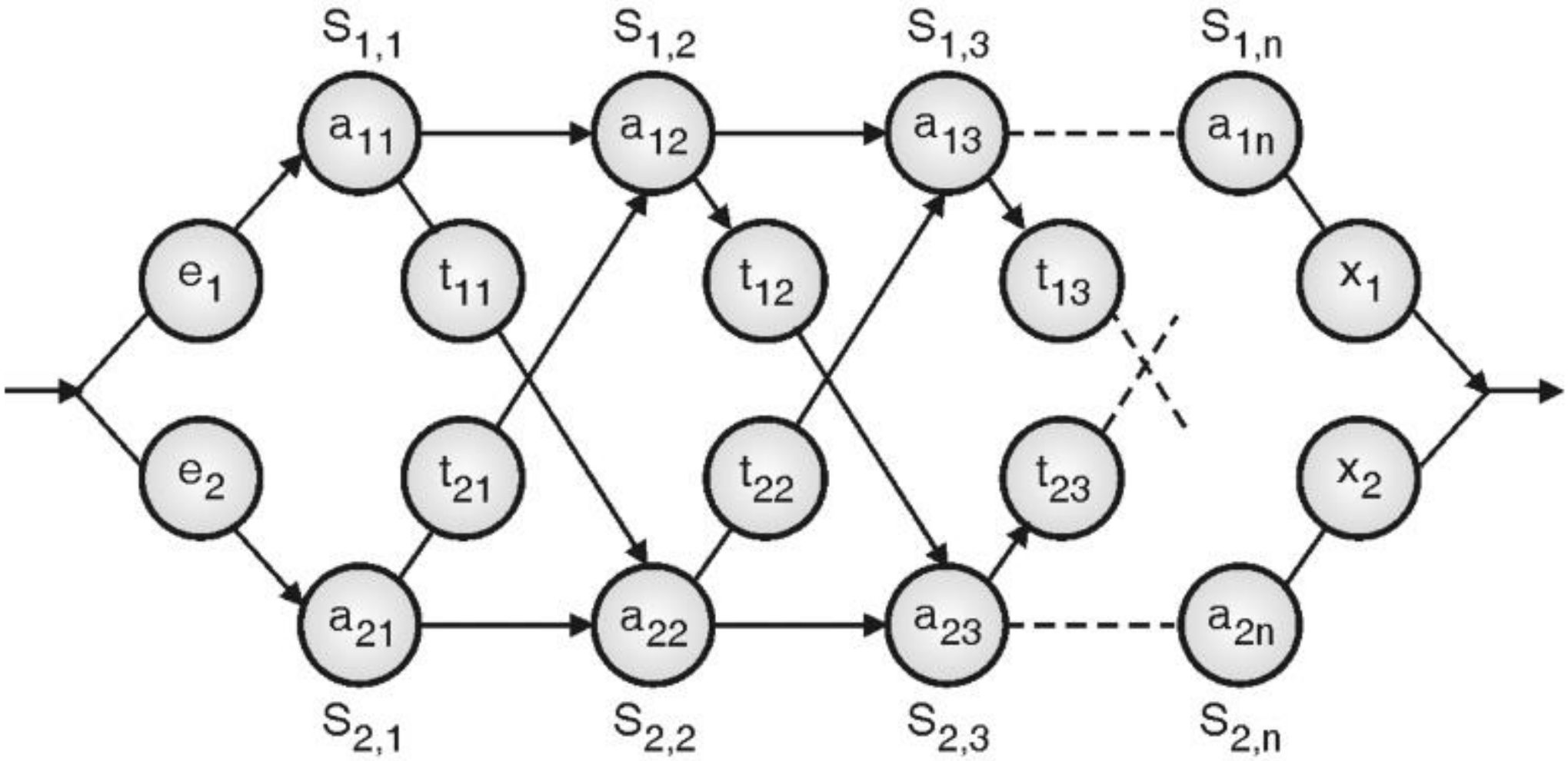
BCSE204L Design and Analysis of Algorithms

Module-2
Topic-4: Assembly Line Scheduling

Assembly Line Scheduling



Assembly Line Scheduling

- A factory has two assembly lines
- Each line has n stations $S_{1,1}$ to $S_{1,n}$ and $S_{2,1}$ to $S_{2,n}$ which builds partial products
- $S_{1,j}$ and $S_{2,j}$ perform the same function but can take different times $a_{1,j}$ and $a_{2,j}$
- Entry times are e_1 and e_2
- Exit times are x_1 and x_2

Assembly Line Scheduling

- After going through a station a partial product can stay on the same line with no cost

or

- Transfer to the other line: Cost of transferring after $S_{i,j}$ is $t_{i,j}$
- Can be transferred to the other line if the load on the current line is high

Assembly Line Scheduling Problem

- Determine which station should be selected from assembly line 1 and which to choose from assembly line 2 in order to minimize the total time to build the entire product.

Recursive Solution:

Product at station $S_{1,j}$ can come either from station $S_{1,j-1}$ or station $S_{2,j-1}$ (Since, the tasks done by $S_{1,j}$ and $S_{2,j}$ are same). But if it comes from $S_{2,j-1}$, it additionally incurs the transfer cost to change the assembly line. Thus, the recursion to reach the station j in assembly line i are as follows:

- Base case(Entry Time)

$$\mathrm{f}_i[j] = e_i + a_{i1}; (if\ j == 1), i = 1, 2$$

Recursive case

- $j > 1$ (Intermediate Times)
- $f_1[j] = min\left(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j}\right)$
- $f_2[j] = min\left(f_2[j-1] + a_{2,j}, f_1[j-1] + t_{1,j-1} + a_{2,j}\right)$

- Exit time (Objective function)
- $f^* = min\left(f_1[n] + x_1, f_2[n] + x_2\right)$

Why Dynamic Programming ?

The given problem is an optimization problem and the above recursion exhibits the overlapping sub problems. Since, there are two ways to reach station $S_{1,j}$, the minimum time to leave station $S_{1,j}$ can be calculated by finding the minimum time to leave the previous two stations, $S_{1,j-1}$ and $S_{2,j-1}$. Since this problem exploits both optimal substructure property and overlapping subproblems property, it is a candidate problem for dynamic programming.

Dynamic Programming Algorithm

1. Characterize the structure of an optimal solution
   - Fastest time through a station depends on the fastest time on previous stations
2. Recursively define the value of an optimal solution
   - $f_1[j] = \min(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j})$
3. Compute the value of an optimal solution in a bottom-up fashion
   - Fill in the fastest time table in increasing order of j (station #)
4. Construct an optimal solution from computed information
   - Use an additional table to help reconstruct the optimal solution

1. $f_1[1] \leftarrow e_1 + a_{1,1}$
2. $f_2[1] \leftarrow e_2 + a_{2,1}$

} Compute initial values of f_1 and f_2

3. **for** $j \leftarrow 2$ **to** n
4. **do if** $f_1[j-1] + a_{1,j} \leq f_2[j-1] + t_{2,j-1} + a_{1,j}$
5. **then** $f_1[j] \leftarrow f_1[j-1] + a_{1,j}$
6. $l_1[j] \leftarrow 1$
7. **else** $f_1[j] \leftarrow f_2[j-1] + t_{2,j-1} + a_{1,j}$
8. $l_1[j] \leftarrow 2$

} Compute the values of $f_1[j]$ and $l_1[j]$

9. **if** $f_2[j-1] + a_{2,j} \leq f_1[j-1] + t_{1,j-1} + a_{2,j}$
10. **then** $f_2[j] \leftarrow f_2[j-1] + a_{2,j}$
11. $l_2[j] \leftarrow 2$
12. **else** $f_2[j] \leftarrow f_1[j-1] + t_{1,j-1} + a_{2,j}$
13. $l_2[j] \leftarrow 1$

} Compute the values of $f_2[j]$ and $l_2[j]$

14. **if** $f_1[n] + x_1 \le f_2[n] + x_2$
15. **then** $f^* = f_1[n] + x_1$
16. $l^* = 1$
17. **else** $f^* = f_2[n] + x_2$
18. $l^* = 2$

Compute the values of the fastest time through the entire factory

---

Alg.: PRINT-STATIONS(l, n)

```
i ← l*
print "line " i ", station " n
for j ← n downto 2
    do i ← l_i[j]
    print "line " i ", station " j - 1
```

line 1, station 5

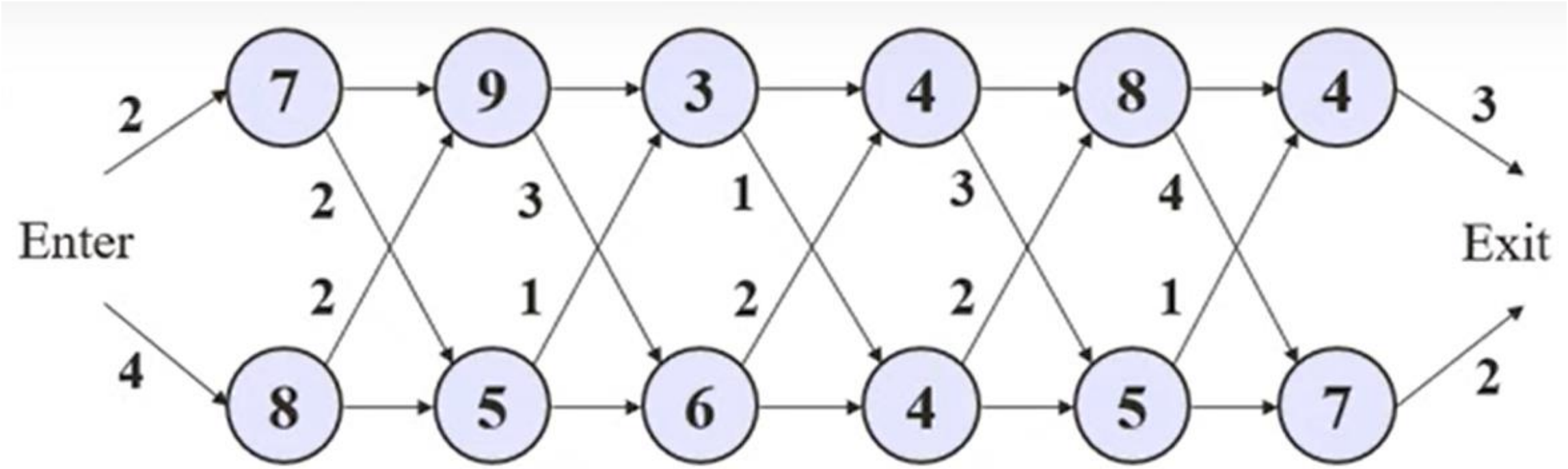
line 1, station 4

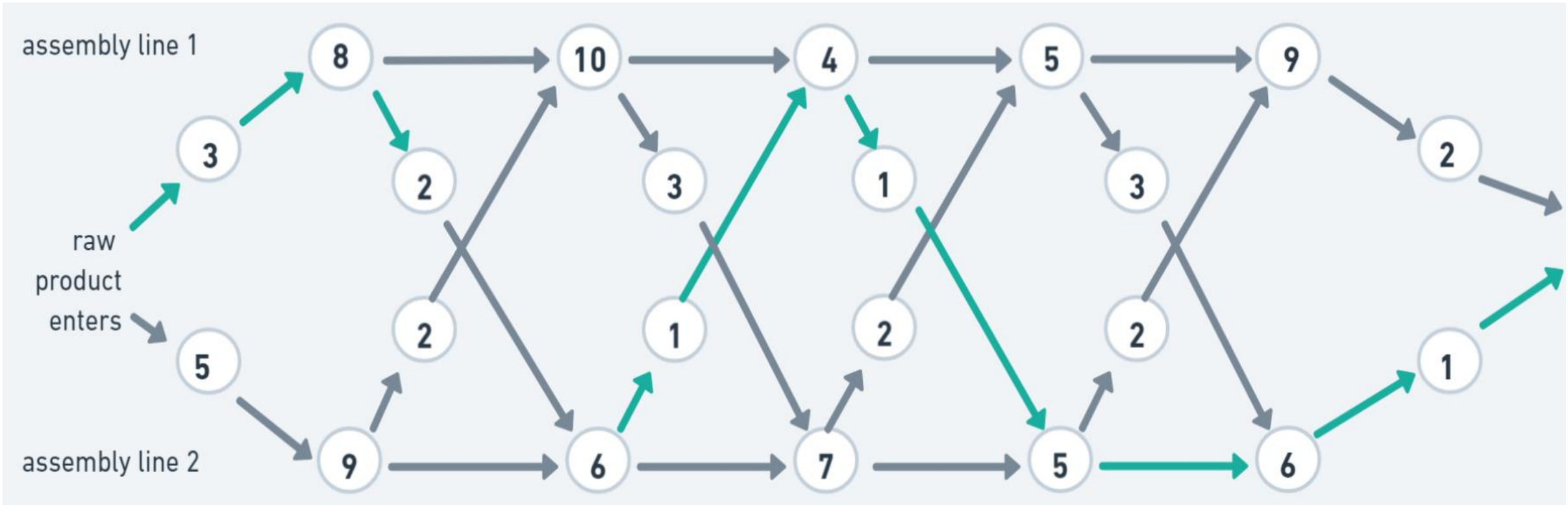
line 1, station 3

line 2, station 2

line 1, station 1

Find Solution:




assembly line 1
8
10
4
5
9
3
2
3
1
3
2
raw
product
enters
2
1
2
2
1
5
9
6
7
5
6
assembly line 2

Pseudocode-1

```
Algorithm ASSEMBLY_LINE_SCHEDULING(n, e, a, t, x)
// n is number of stations on both assembly
// e is array of entry time on assembly
// a is array of assembly time on given station
// t is array of the time required to change assembly
line
//x is array of exit time from assembly


f1[1] ← e1 + a1,1
f2[1] ← e2 + a2,1
```

Pseudocode-2

```
for j ← 2 to n do
    if f1[j - 1] + a1, j  ≤ f2[j - 1] + t[j - 1] + t +
a2, j then
        f1[j] ← f1[j - 1] + a1, j
        L1[j] ← 1
    else
        f1[j] ← f2[j - 1] + t2,j - 1+ a1, j
        L1[j] ← 2
    end
```

Pseudocode-3

```
    if f2[j − 1] + a2, j ≤ f1[j − 1] + t1, j − 1 + a2,
j then
        f2[j] ← f2[j − 1] + a2, j
        L2[j] ← 2
    else
        f2[j] ← f1[j − 1] + t1, j − 1 + a2, j
        L2[j] ← 1
    end
```

Pseudocode-4

```
    if f1[n] + x1  ≤ f2[n] + x2 then
        F* ← f1[n] + x1
        L* ← 1
    else
        F* ← f2[n] + x2
        L* ← 2
    end
end
```

Pseudocode- Print Stations

```
Algorithm  PRINT_STATIONS(l, n)


i ← 1*
print "line " i ", station " n
for j ← n downto 2 do
    i ← Li[ j ]
    print "line " i ",station " j - 1
end
```

Analysis

- A single for loop
- So, $O(n)$

Thank you..